

Programación Orientada a Objetos

Documentación Técnica — Trabajo Práctico Integrador

UNQ-Shop

Grupo 6

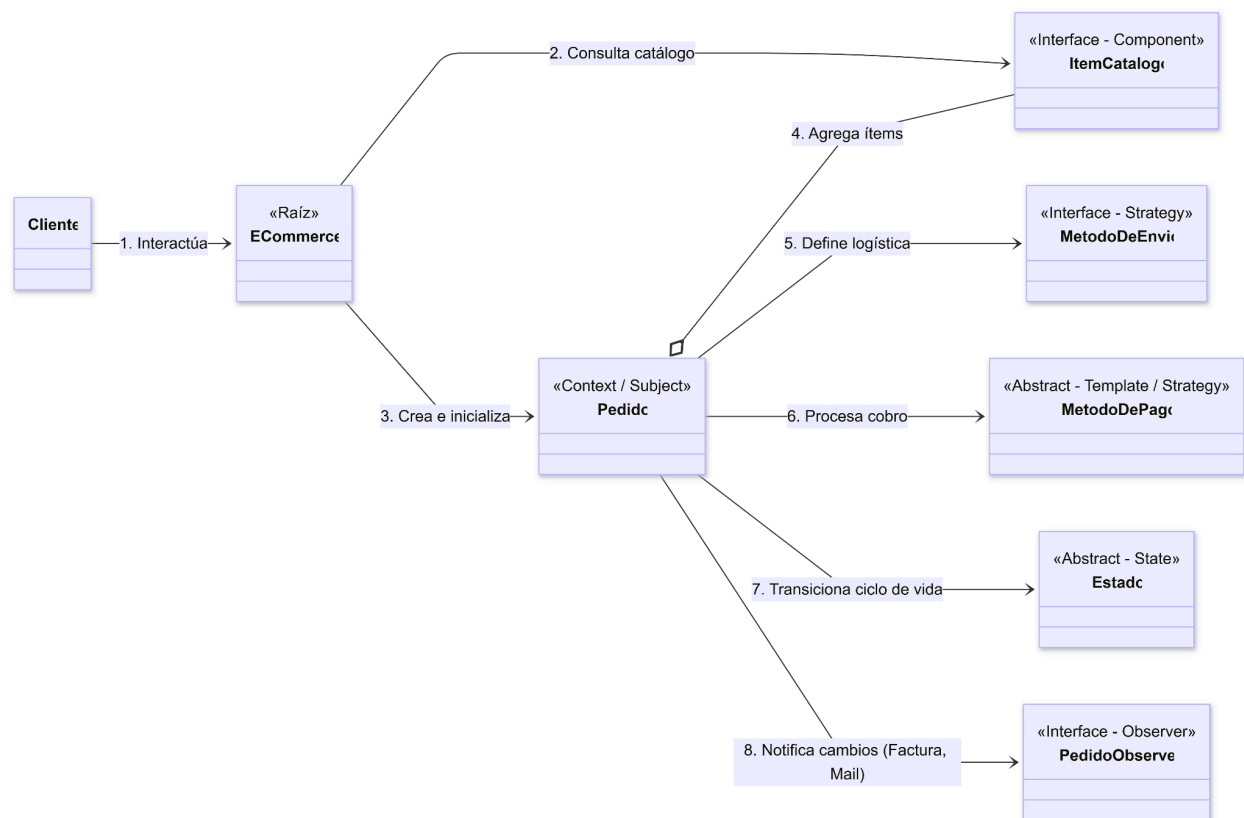
Integrante	Email
Ferreya, Amanda	amandac4e@gmail.com
Pastorini, Renata	pastorinirenata@gmail.com
Peralta, Mariela	marielajperalta@gmail.com

Introducción

En este documento describimos las decisiones de diseño tomadas para el desarrollo del E-commerce correspondiente al trabajo final. El mismo fue desarrollado siguiendo los patrones especificados en Gamma et al. (GoF), los cuales detallaremos más adelante.

Dividimos el documento siguiendo las consignas para una mejor comprensión del avance progresivo del proyecto. En cada sección se detallan las decisiones de diseño tomadas, las dudas que fueron surgiendo y cómo las resolvimos, y otros detalles de implementación que nos resulten importantes de explicar.

Flujo de la Información



En este diagrama de flujo de información simplificado del dominio se ilustra la secuencia lógica de interacciones y dependencias entre los componentes principales del sistema durante el ciclo de vida de una orden, omitiendo detalles de implementación para priorizar la visibilidad del flujo.

1. Punto de Entrada (ECommerce): Es el orquestador principal del dominio. Contiene el catálogo global y el registro de todos los pedidos. Es la puerta de entrada para interactuar con el sistema.
2. Armado del Carrito (Pedido e ItemCatalogo): Cuando se inicia una compra, el ECommerce instancia un Pedido. Este pedido comienza a agrupar objetos que respetan la interfaz ItemCatalogo (ya sean productos sueltos o paquetes).

3. Resolución Comercial (MetodoDePago y MetodoDeEnvio): Una vez que el pedido tiene sus ítems, la información fluye hacia las estrategias de negocio. El pedido le delega a MetodoDeEnvio el cálculo del costo logístico y, posteriormente, le entrega su información a MetodoDePago para validar y ejecutar la transacción.
4. Ciclo de Vida (Estado): Durante todo este proceso, el flujo de información está condicionado por el patrón State. El Pedido transiciona delegando su comportamiento a la clase Estado abstracta (pasando de Borrador a Confirmado, En Preparación, etc.).
5. Reacción del Sistema (PedidoObserver): Finalmente, cuando el pedido transiciona de estado (por ejemplo, el pago es exitoso y el estado cambia), la información fluye hacia afuera de forma pasiva. El pedido actúa como "Subject" y dispara notificaciones hacia los "Observers" , permitiendo que módulos independientes generen facturas, envíen correos o sumen puntos sin acoplarse al núcleo.

Desarrollo

2.1 — Catálogo de productos

Para el modelado del catálogo, la decisión que tomamos fue la aplicación del patrón **Composite**. Esto nos permitió establecer una estructura de árbol que trata de forma uniforme tanto a los productos individuales como a los paquetes (*ItemCatalogo*)..

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Cómo logramos que el sistema (*Pedido* y *ECommerce*) no tenga condicionales para distinguir si opera sobre un producto suelto o un combo?**
 - Definimos la interfaz genérica *ItemCatalogo* que establece el contrato comercial básico: obtener precio final, consultar stock, conocer el peso, entre otros. De esta interfaz implementan directamente *Producto* y *Paquete*, y polimórficamente responden a los mismos mensajes.
- **¿Cómo se administra el stock de un *Paquete*? ¿Se calcula dinámicamente en base al stock de sus componentes o tiene existencia propia ?**
 - Decidimos que el *Paquete* administre su propio stock físico de manera independiente mediante el atributo *Integer stock*. Modelamos el concepto de paquete como un conjunto de productos pre-armado. Al invocar *aumentarStock(cantidad)* sobre un *Paquete*, el sistema incrementa el stock del paquete, pero recorre su mapa de ítems internos para decrementar el stock de los componentes que conforman ese paquete (*item.decrementarStock(cantidad * cant)*), y estos pasan a estar “en reserva” dentro de los paquetes.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce

Composite (Estructura Catálogo)	Component	ItemCatalogo	Establece la interfaz base común para calcular precios, nombres y gestionar stock, unificando el trato a ítems simples o combos.
	Leaf	Producto	Representa un artículo unitario básico del inventario que no contiene otros componentes.
	Composite	Paquete	Representa un paquete que almacena una lista interna de ItemCatalogo y calcula de forma agregada el precio y stock.

2.2 — Ciclo de vida del pedido

Para administrar el ciclo de vida de un `Pedido` y sus múltiples transiciones (Borrador, Confirmado, En Preparación, Enviado, Entregado y Cancelado), implementamos el patrón **State**. El comportamiento comercial y las restricciones operativas varían polimórficamente delegando la responsabilidad en objetos de estado concretos.

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Quién debe tener el control de las transiciones y conocer cuál es el "siguiente estado válido"?**
¿El contexto (`Pedido`) o los estados concretos?
 - Decidimos que las clases de estado concretas controlen la lógica de transición. Para lograrlo, los métodos de la clase abstracta `Estado` reciben como parámetro la referencia del contexto (`Pedido`). Cuando una operación es válida, el estado concreto ejecuta de forma directa `pedido.setEstado(nuevoEstado)`, manteniendo a `Pedido` desacoplado de las transiciones esperadas.
- **¿Cómo gestionamos de forma limpia las operaciones inválidas de un estado sin duplicar lógica?**
 - En lugar de obligar a cada estado concreto a implementar validaciones para todas las acciones prohibidas, definimos el comportamiento por defecto en la clase abstracta `Estado`. Creamos el método protegido `errorDeTransicion` que lanza de manera genérica la excepción `TransicionDeEstadoInvalidaException`. Los estados concretos sólo sobrescriben los métodos correspondientes a sus transiciones válidas; cualquier otra invocación hereda automáticamente el comportamiento de falla.
- **¿Cómo logramos limitar la funcionalidad de agregar o quitar ítems una vez que el pedido avanza de la etapa inicial?**

- Añadimos el mensaje `validarSiPuedoAgregarOSacarItems()` al contrato de `Estado`, e hicimos que los métodos `addItem()` y `removeItem()` en `Pedido` lo invoquen como paso previo obligatorio. La clase abstracta `Estado` lanza una excepción por defecto para este método. Únicamente la clase `EstadoBorrador` sobrescribe esta lógica permitiendo que la operación continúe con éxito.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
State	Context	Pedido	Mantiene la referencia al estado actual del ciclo de vida del pedido y le delega el comportamiento comercial y las transiciones.
	State	EstadoPedido (Abstracta)	Define la interfaz de comportamiento común y las transiciones válidas e inválidas permitidas para todos los estados.
	ConcreteState	Borrador, Confirmado, EnPreparacion, Enviado, Entregado, Cancelado	Implementan la lógica de negocio y las restricciones específicas de cada etapa del ciclo de vida de un pedido.

2.3 — Métodos de envío

Para resolver la variabilidad en los costos y tiempos de envío, implementamos el patrón **Strategy**. Definimos la interfaz `MetodoDeEnvio` como contrato general, permitiendo que la clase `Pedido` delegue el cálculo sin conocer las reglas logísticas específicas. Como las librerías externas brindadas tienen firmas incompatibles para los envíos a domicilio, aplicamos también el patrón **Adapter**, integrando estas APIs ajenas a nuestro dominio.

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Cómo evitamos que la clase `Pedido` concentre múltiples algoritmos de cálculo de envío (estándar, express, sucursal) rompiendo el principio de Responsabilidad Única?**
 - Aplicamos el patrón `Strategy`. Extraemos la lógica a la interfaz `MetodoDeEnvio`, la cual expone los métodos `calcularCostoDeEnvio(pedido)` y `estimacionDeDias()`. El `Pedido` mantiene una referencia a esta interfaz y le delega la responsabilidad, lo que nos habilita a agregar nuevos métodos de envío a futuro sin modificar la lógica de esta clase.
- **¿Cómo integramos las clases provistas (`CorreoArgentina` y `EnvioExpress`) si sus firmas y parámetros no coinciden con nuestra interfaz `MetodoDeEnvio`?**
 - En lugar de ensuciar nuestro modelo modificando el contrato para que encaje con las APIs externas, implementamos el patrón `Adapter`.
 - Construimos las clases `EnvioEstandarAdapter` y `EnvioExpressAdapter`. Estas clases implementan nuestra interfaz `MetodoDeEnvio` y por dentro encapsulan una instancia de la librería externa correspondiente. Cuando el pedido pide calcular el costo, el `Adapter` extrae los datos necesarios (como el peso o el monto total del pedido) y los traduce al formato que la librería espera (por ejemplo: `apiEnvio.estimarEnvio(peso, direccion)`).
- **¿Qué sucede con el retiro en sucursal, que es una lógica interna y no requiere una API externa?**
 - Creamos la clase `RetiroEnSucursal` que implementa directamente `MetodoDeEnvio` como una `ConcreteStrategy` pura. Para el `Pedido` es indistinto si el cálculo lo está realizando un adaptador hacia una API externa o un algoritmo propio del sistema; en ambos casos interactúa polimórficamente con el mismo contrato.
- **Cantidad de días de entrega.**
 - En un principio la estimación de días para "Retiro en sucursal" se resolvía con un `System.out.println`. Se corrigió para que `estimacionDeDias()` retorne el string en lugar de imprimirlo.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
Strategy	Context	Pedido	Delega el cálculo de costos y tiempos de entrega en la estrategia de logística configurada en el pedido.
	Strategy	MetodoDeEnvio (Interfaz)	Define la interfaz requerida para cualquier tipo de logística o despacho de

			mercadería.
	ConcreteStrategy	EnvioExpressAdapter, EnvioEstandarAdapter, RetiroEnSucursal	Calculan de manera independiente los tiempos y costos según la modalidad logística seleccionada.
Adapter	Target	MetodoDeEnvio	La interfaz de dominio interno requerida por el e-commerce para procesar despachos.
	Adapter	EnvioExpressAdapter, EnvioEstandarAdapter	Traducen e interconectan los métodos del contrato de nuestro dominio (Target) a las firmas nativas de las librerías externas (Adaptee).
	Adaptee	EnvioExpress, CorreoArgentina	Clases externas encapsuladas (dentro de .libreriasExternas) con contratos incompatibles con nuestro sistema comercial.

2.4 — Métodos de pago

Para resolver el procesamiento de los pagos garantizando que todas las transacciones sigan estrictamente la misma secuencia operativa obligatoria, combinamos los patrones **Template Method** y **Strategy**. Definimos `MetodoDePago` como una clase abstracta que funciona como la estrategia general de cobro para el `Pedido` y que, al mismo tiempo, establece un método plantilla para fijar el algoritmo de ejecución.

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Cómo garantizamos que cualquier medio de pago (tarjeta, transferencia o billetera) ejecute los pasos de validación, reserva, ejecución y notificación exactamente en ese mismo orden sin que las subclases alteren el flujo?**
 - Aplicamos el patrón **Template Method**. En la clase abstracta `MetodoDePago`, definimos el método `procesarPago(pedido: Pedido)` con el modificador `final`. Este esqueleto define de forma inmutable la secuencia exacta de llamadas : primero `validarDatos(pedido)`, luego `reservarFondos(pedido)`, seguido por `ejecutarTransaccion(pedido)` y finalmente `notificarResultado(pedido)`. Las subclases no pueden alterar el orden, solo completar las piezas faltantes.
- **¿Cómo manejamos la diferencia entre los pasos que son obligatorios y específicos para cada tipo de pago frente a las operaciones que admiten un comportamiento común por defecto?**
 - En la clase abstracta `MetodoDePago`, declaramos como métodos abstractos a `validarDatos()`, `reservarFondos()` y `ejecutarTransaccion()`, forzando a cada método de pago a proveer su propia lógica obligatoria según sus respectivas APIs externas. Por otro lado, definimos a `notificarResultado()` como un método concreto con una implementación base (el registro del código o comprobante de la transacción), actuando como un *Hook* que las subclases pueden sobrescribir opcionalmente si requieren una lógica especializada (como la notificación push de la billetera).
- **¿Cómo se conecta este módulo con el resto del sistema del e-commerce sin generar un acoplamiento rígido con las estrategias de pago concretas?**
 - Reutilizamos la abstracción de `MetodoDePago` bajo el rol de *Strategy* abstracta. La clase `Pedido` solo mantiene una referencia hacia este tipo abstracto a través de su atributo `metodoPago`. Al momento de abonar, el pedido invoca polimórficamente a `metodoPago.procesarPago(this)`, delegando por completo el comportamiento financiero sin importar cuál es efectivamente el método de pago elegido por el cliente.
- **Por qué la API se recibe en el constructor.**
 - Cada medio de pago recibe su interfaz externa (`ApiTransferenciaBancaria`, `ApiBilleteraVirtual`, `ApiTarjetaDeCredito`) por constructor en vez de instanciarla internamente. La alternativa sería que, por ejemplo, `PagoConTarjeta` haga `new ApiMercadoPagoReal()` — eso acopla el dominio a un proveedor concreto y viola el principio Open/Closed. Al inyectar la interfaz, `PagoConTarjeta` no sabe ni le importa qué implementación real recibe, mientras cumpla el contrato.

Decisiones de diseño:

Opción de diseño descartada: Exploramos la idea de tener un único "ejecutor" por medio de pago (una sola instancia procesando todas las transacciones de ese tipo) que recibiera credenciales genéricas. Se descartó porque los datos de cada medio no tienen nada en común y obligaba a castear

constantemente. Resolvimos volver al diseño más simple donde cada operación de pago genera una instancia nueva del medio elegido.

Comprobantes: Cada medio de pago genera y guarda su propio tipo de comprobante ([CuponDePago](#), [ComprobanteTransferencia](#)) durante `notificarResultado()`. La billetera virtual es la excepción, ya que el requerimiento pide únicamente notificación push, por lo que no genera comprobante.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
Strategy	Context	ECommerce / Pedido	Utiliza e invoca el algoritmo de pago seleccionado por el cliente para procesar la transacción del pedido.
	Strategy	MetodoDePago (Abstracta)	Define el contrato genérico para procesar y validar transacciones financieras dentro de la plataforma.
	ConcreteStrategy	Tarjeta, Transferencia, Billetera	Implementan las particularidades del procesamiento y reglas de negocio propias de cada medio de cobro.
Template Method	AbstractClass	MetodoDePago	Define el algoritmo estructural inmutable mediante <code>procesarPago()</code> (marcado como final), ordenando rígidamente los pasos de validación,

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
			reserva, ejecución y notificación.
	ConcreteClass	Tarjeta, Transferencia, Billetera	Completan los pasos abstractos obligatorios requeridos por el esqueleto del algoritmo de pago y heredan los hooks por defecto.

2.5 — Notificaciones del pedido

Para responder al requerimiento de que múltiples subsistemas independientes reaccionen ante los cambios de estado de un pedido sin que el módulo central de pedidos tenga que conocerlos o depender de ellos, implementamos el patrón **Observer**. Esto garantiza que el e-commerce sea completamente extensible ante la futura incorporación de nuevos módulos que requieran escuchar estos eventos.

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Cómo logramos que el módulo de pedidos comunique sus novedades de ciclo de vida sin acoplarse rígidamente a los componentes de mensajería o facturación?**
 - Establecemos que la clase `Pedido` asume el rol de *Subject*, administrando una lista interna de suscriptores basada en la interfaz abstracta `PedidoObserver`. Al cambiar su estado interno, el pedido recorre polimórficamente la lista de observadores y dispara los avisos correspondientes, delegando la responsabilidad de la reacción en cada componente externo de forma desacoplada.
- **¿Cómo manejamos la restricción de que ciertos subsistemas sólo actúen ante transiciones específicas (como el mail en las etapas confirmada, enviada y entregada o fidelización únicamente ante cancelaciones) sin obligar a las clases a implementar métodos vacíos manualmente?**
 - En la interfaz `PedidoObserver`, definimos métodos con implementaciones por defecto vacías (*hooks*). De esta forma, `NotificadorDeEmail`, `GeneradorDeFactura` y `Fidelizador` solo sobrescriben estrictamente los métodos vinculados a los eventos de su interés, ignorando el resto.

- ¿Cómo se vinculan los servicios externos de mensajería provistos, como `MailSender`, sin alterar la pureza de la lógica del dominio?
- Encapsulamos el uso de estas interfaces externas directamente dentro de los observadores concretos. Por ejemplo, `NotificadorDeEmail` recibe la interfaz `MailSender` como una dependencia en su constructor para despachar los correos informativos al cliente. Así, `Pedido` se desentiende por completo de las direcciones de destino, títulos o adjuntos, concentrándose únicamente en su flujo operativo de negocio.

Decisiones de diseño:

Creamos la clase `DatosCliente` (email + teléfono) para que el constructor de `Pedido/ECommerce.newPedido()` no crezca cada vez que se suma un nuevo canal de notificación.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
Observer	Subject	Pedido	Registra, remueve y notifica a la colección de observadores cuando se produce un cambio en su estado interno.
	Observer	PedidoObserver (Interfaz)	Define el contrato de escucha general con métodos vacíos (hooks) para que los observadores solo implementen lo que necesiten.
	ConcreteObserver	NotificadorDeEmail, Fidelizador, GeneradorDeFactura	Reaccionan ante eventos del pedido ejecutando tareas secundarias desconectadas del núcleo del dominio (envío de correos, sumas de puntos, facturación).

2.6 — Búsqueda en el catálogo

Para resolver los requerimientos de filtrado del catálogo de manera flexible y permitir la creación de consultas complejas sin acoplar la estructura del sistema, decidimos combinar los patrones **Strategy** y **Composite**. Definimos la clase abstracta `CriterioDeBusqueda` que funciona como la estrategia base para cualquier filtro. A su vez, esta abstracción asume el rol de *Component* en un segundo diseño *Composite*, lo que permite que las compuertas lógicas agrupen y combinen filtros atómicos para formar árboles de búsquedas booleanas anidadas.

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Cómo logramos que el módulo de búsqueda en `ECommerce` no tome decisiones sobre cómo evaluar los ítems ni se llene de condicionales cada vez que se agrega un filtro nuevo?**
 - Aplicamos el patrón **Strategy**. El método `filtrar` en `ECommerce` recibe un único argumento de tipo abstracto `CriterioDeBusqueda`. La responsabilidad de evaluar si un artículo califica o no, recae completamente en la estrategia recibida a través del mensaje polimórfico `cumple(item)`. De esta manera, si el negocio exige filtrar por un criterio nuevo en el futuro, solo hace falta crear una nueva subclase sin alterar la lógica del e-commerce.
- **¿Cómo permitimos que el cliente combine libremente criterios simples (nombre, precio, categoría, disponibilidad) para construir consultas compuestas de cualquier profundidad?**
 - Utilizamos el patrón **Composite**. Diseñamos las compuertas lógicas `CriterioAnd`, `CriterioOr` y `CriterioNot` heredando directamente de `CriterioDeBusqueda`. Al ser componentes de la misma jerarquía, las compuertas lógicas pueden almacenar colecciones de otros criterios, ya sean filtros atómicos simples o incluso otras compuertas anidadas. La evaluación se resuelve de forma recursiva en todo el árbol de condiciones.
- **¿Cómo se traslada la responsabilidad de armar la consulta compleja al cliente del sistema?**
 - El cliente del dominio configura y anida el objeto compuesto antes de invocar la búsqueda. `ECommerce` recibe la raíz de ese árbol de criterios y dispara la evaluación.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
Composite (Filtros de Búsqueda)	Component	CriterioDeBusqueda (Interfaz)	Establece el método común de filtrado (evaluar(ItemCatalogo)) aplicable sobre los artículos del catálogo.
	Leaf	FiltroPorNombre, FiltroPorPrecioMaximo, FiltroPorCategoria, FiltroPorDisponibilidad	Filtros atómicos simples encargados de evaluar una sola condición de negocio a la vez.
	Composite	CriterioAnd, CriterioOr, CriterioNot	Compuertas lógicas complejas que agrupan otros criterios permitiendo componer árboles de búsquedas booleanas anidadas e infinitas.
Strategy	Context	ECommerce	Invoca el algoritmo de filtrado seleccionado a través del método <code>filtrar(CriterioDeBusqueda)</code> .
	Strategy	CriterioDeBusqueda (Abstracta)	Define el contrato genérico <code>cumple(item)</code> para cualquier algoritmo de validación o filtro en el catálogo.

	ConcreteStrategy	<pre>FiltroPorNombre, FiltroPorPrecioM aximo, FiltroPorCategor ia, FiltroPorDisponi bilidad</pre>	Implementan los algoritmos de filtrado atómicos y específicos para comprobar las condiciones de los ítems.
--	------------------	---	--

2.8 — Reportes

El sistema requería la capacidad de generar reportes sobre el estado del negocio, con la particularidad de que cada reporte debía poder exportarse en más de un formato de salida, específicamente texto plano, CSV y HTML. Para cumplir con esto y evitar acoplar la lógica de presentación a las clases de dominio, implementamos el patrón **Visitor**. Esto nos permite encapsular las reglas de formato en clases independientes, manteniendo limpias a las entidades del catálogo y permitiendo agregar nuevos formatos a futuro sin modificar el modelo existente.

Preguntas y resoluciones surgidas durante el desarrollo:

- **¿Cómo logramos exportar los datos de los ítems en distintos formatos (TXT, CSV, HTML) sin ensuciar las clases **Producto** y **Paquete** con código de presentación?**
 - Extraemos la responsabilidad de "dar formato" hacia la interfaz `ReporteVisitor`. Las clases de nuestro catálogo solo implementan un método `aceptar(ReporteVisitor visitor)`, el cual simplemente delega la acción ejecutando `visitor.visitar(this)`. Las clases concretas `ReporteTxt`, `ReporteCSV` y `ReporteHTML` son las encargadas de extraer los datos y concatenar las cadenas de texto con las etiquetas o separadores correspondientes.
- **¿Cómo garantizamos que el reporte procese correctamente la información sabiendo que los **Productos** simples y los **Paquetes** tienen estructuras distintas?**
 - Aprovechamos la técnica de *Double Dispatch* inherente al patrón Visitor. La interfaz `ReporteVisitor` define firmas separadas: `visitar(Producto producto)`

y `visitar(Paquete paquete)`. Cuando llamamos a `aceptar()` sobre una colección polimórfica de `ItemCatalogo`, el lenguaje resuelve dinámicamente en tiempo de ejecución cuál de los métodos `visitar` sobrecargados debe ejecutar, evitando por completo el uso de validaciones de tipo engorrosas como `instanceof`.

- **¿Cómo recolectamos y cruzamos los datos estadísticos, como las cantidades vendidas y los promedios de precios, si el catálogo solo conoce su estructura estática?**
 - Diseñamos la clase `ReporteVentas` que interactúa con la raíz `ECommerce` para procesar el historial de los pedidos. Esta estructura recolecta la información y la empaqueta en instancias de `ResumenVentaItem`. Luego, inyecta esta data estadística en los visitantes concretos mediante el método `prepararResumen(resumen)` instantes antes de hacerlos visitar el ítem, logrando fusionar la estadística de ventas con la estructura jerárquica del catálogo de forma elegante.

Se investigó e implementó Visitor para desacoplar el recorrido del catálogo de los tres formatos de salida exigidos (texto, CSV, HTML). `ItemCatalogo` expone `aceptar(ReporteVisitor)` como `Element`; `Producto` y `Paquete` son los `ConcreteElement` que disparan el `double dispatch`; `ReporteTxt/ReporteCSV/ReporteHTML` son los tres `ConcreteVisitor`. El núcleo del reporte se calcula una sola vez en `ECommerce.cantidadDeVentasYPreciosPromedioPorItem()`, independiente del formato; cada `Visitor` solo decide cómo presentar esos datos ya calculados.

Aunque hoy `visitar(Producto)` y `visitar(Paquete)` tienen un comportamiento muy similar en los tres formatos, se mantuvo el `double dispatch` en lugar de resolverlo con un `instanceof` — es un punto de extensión limpio y evita el antipatrón de preguntar por tipo.

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
Visitor	Visitor	ReporteVisitor (Interfaz)	Declara los métodos de visita específicos aplicables sobre los elementos concretos del catálogo comercial.
	ConcreteVisitor	ReporteTxt, ReporteCsv,	Implementan el procesamiento de datos y la exportación al formato correspondiente del

Patrón GoF	Rol según Gamma et al.	Clase / Interfaz	Responsabilidad en el E-Commerce
		ReporteHtml	reporte de productos más vendidos.
	Element	ItemCatalogo	Define la operación de doble despacho mediante el método abstracto accept(ReporteVisitor).
	ConcreteElement	Producto, Paquete	Implementan el método accept redirigiendo el flujo hacia el visitante con la firma exacta del elemento (v.visit(this)).